📑 Software Requirement Specification (SRS)

Project Name: Gateway OTP (Next-Generation Smart OTP Verification Gateway)

---

1. Project Overview & Objective

Gateway OTP is a highly scalable, production-ready, and cost-effective multi-channel One-Time Password (OTP) Verification Gateway backend engine. The core system architecture is custom-tailored for **AI-driven "Vibe Coding"** methodologies and rapid micro-caching standards.

Core Objective: To completely eliminate infrastructure delivery and communication overhead costs for early-stage startups and small-to-medium software teams. Gateway OTP features a dynamic cost-optimization routing layer that evaluates channel availability and operational thresholds in real-time to dispatch security tokens via the most cost-efficient medium possible (WhatsApp $ ightarrow$ Email $ ightarrow$ Fallback SMS).

2. Core Architectural Features

- **Cryptographically Secure Token Generation:** The system rejects pseudo-random generators (such as `Math.random()`) due to vulnerability against predictability and guessing attacks. Instead, it utilizes the native Node.js `crypto` module to generate unguessable, high-entropy 6-digit numeric verification tokens.
- **Smart Auto-Switching Routing Engine:** Implements a cascading waterfall routing mechanism based on cost-efficiency thresholds:
  - **Priority 1 (Primary): Meta WhatsApp Cloud API** — Utilizes the monthly free tier allowance (1,000 user-initiated conversations). System defaults all routing here initially.
  - **Priority 2 (Secondary Fallback): Resend / Brevo Transactional Email APIs** — Automatically shifts traffic to the email channel if WhatsApp thresholds are exhausted or connection drops occur (3,000+ free monthly emails allowance).
  - **Priority 3 (Tertiary Fallback): Third-Party SMS Gateway** — Integrates client-configured telecom nodes strictly as a final fallback layer if telecom billing balances are active; otherwise, exceptions are caught gracefully.
- **Dynamic Micro-Caching Layer:** To eliminate heavy database lookup overheads and disk write bottlenecks, the engine stores volatile operational tokens inside an in-memory cache. All dynamic OTP tokens are bound to a strict Time-To-Live (TTL) expiry window of exactly **300 seconds (5 minutes)**.
- **Anti-Brute Force Protection (Rate Limiting):** Restricts malicious iteration attacks against a single identifier (phone number or email) by enforcing a maximum cap of **3 consecutive failed validation attempts**. Exceeding this cap places the target identifier/IP into a temporary lock down cooldown state for **15 minutes**.

- **Anti-Spam Resend Guardrail:** To mitigate API payload exhaustion and prevent resource billing spam loops, a strict resource lock prevents a generic client from requesting a new token for the same identifier within **60 seconds** of the last successful dispatch.

3. Tech Stack & Infrastructure Design

- **Backend Runtime:** Node.js environment utilizing a clean, asynchronous JavaScript execution context driven by the Express.js application framework.
- **State Management & Caching:** Redis cache layer. Standalone standalone Redis for local development, and serverless distributed `Upstash Redis` for production/staging environments.
- **Cloud Deployment:** Managed via Render.com Web Service architecture utilizing an automated CI/CD deployment pipeline synced directly to the target GitHub repository branch.

4. Database Cache Schema (Redis Key-Value Models)

To guarantee microsecond verification lookups, the cache layers must use the following clean, predictable Key-Value design structures:

A. Active Token Verification Matrix

- **Key Design Pattern:** `auth:otp:[CountryCode][PhoneNumber]`
- **Data Structure Type:** Stringified JSON Map / Hash Matrix
- **Allocation Lifecycle:** Automatically drops via native Redis TTL engine set to exactly 300 seconds.

```
"auth:otp:919876543210": {
    "otp": "584291",
    "attempts": 0,
    "createdAt": 1783647291
}
```

B. Client Request Cool-Down State Lock

- **Key Design Pattern:** `auth:cooldown:[CountryCode][PhoneNumber]`
- **Data Structure Type:** String Binary Bit Flag
- **Allocation Lifecycle:** Automatically drops via native Redis TTL engine set to exactly 60 seconds.

```
"auth:cooldown:919876543210": "active"
```

5. REST API Endpoints Specification

📤 API Endpoint 1: Dispatch Verification Code (Send OTP)

Route HTTP Verb: `POST /api/v1/otp/send`

Transport Headers: `Content-Type: application/json`

Expected Payload Body:

```
{
  "phoneNumber": "919876543210",
  "email": "user@example.com",
  "channelPreference": "smart"
}
```

Expected Success Response Structure (200 OK):

```
{
  "success": true,
  "message": "OTP dispatched successfully via WhatsApp.",
  "requestId": "req_849201"
}
```

📥 API Endpoint 2: Evaluate Verification Token (Verify OTP)

Route HTTP Verb: `POST /api/v1/otp/verify`

Transport Headers: `Content-Type: application/json`

Expected Payload Body:

```
{
  "phoneNumber": "919876543210",
  "userOtp": "584291"
}
```

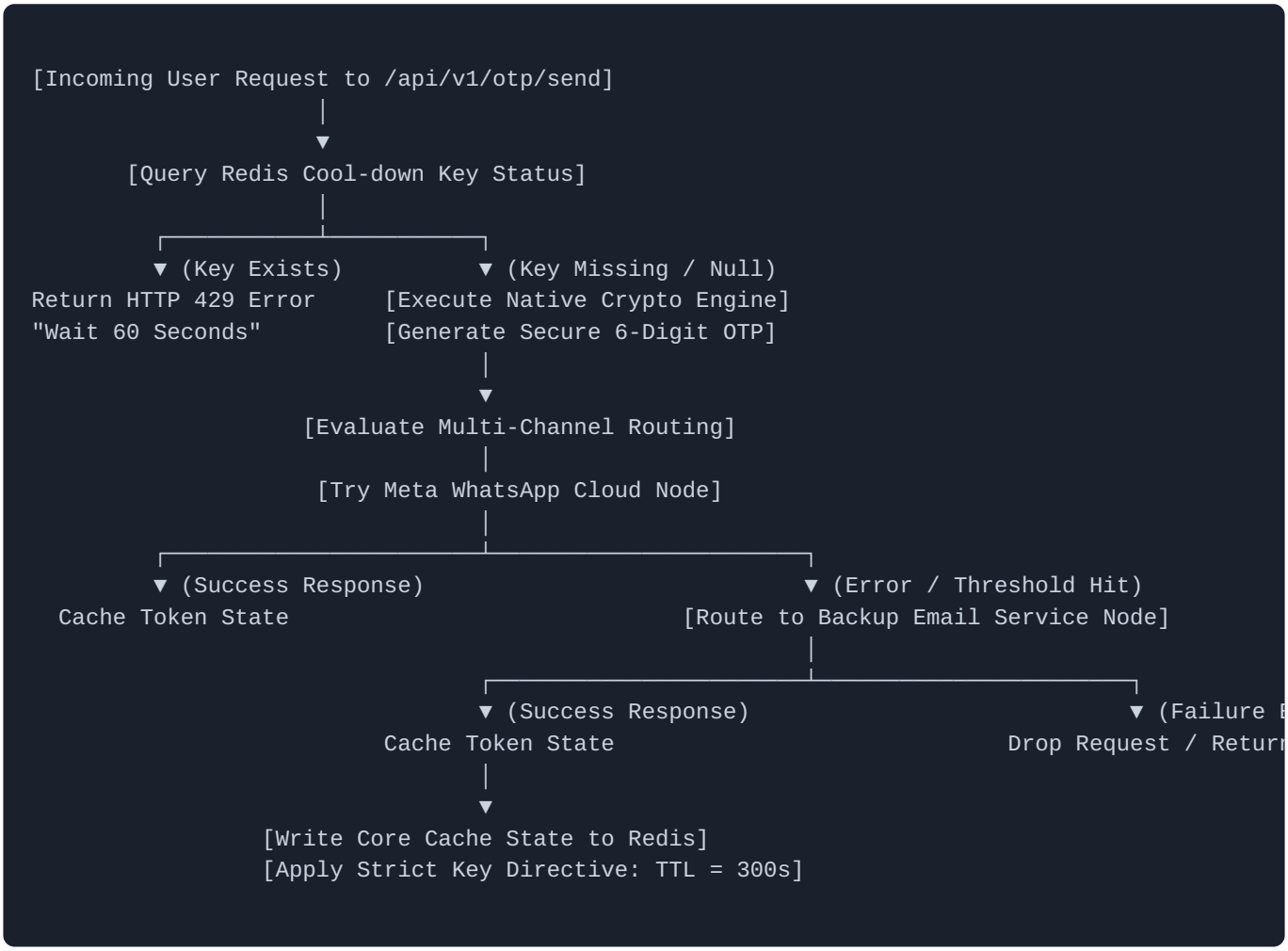
Expected Identity Match Verification Response (200 OK):

```
{
  "success": true,
  "message": "OTP verification successful. Access granted."
}
```

Expected Exception Failure Response (400 Bad Request):

```
{
  "success": false,
  "message": "Invalid OTP. 2 attempts remaining."
}
```

6. System Execution Workflows & Logical Fail-Safe Streams



7. Action Plan for Automated Cursor AI Code Generation

1. Open your system terminal and create your root project directory:
   `mkdir GatewayOTP-Backend && cd GatewayOTP-Backend`
2. Initialize your clean Node Package Manager environment mapping:
   `npm init -y`
3. Inject the exact required application dependencies directly into your workspace node module:
   `npm install express redis dotenv crypto`
4. Create your main runtime backend execution index point file:
   `touch server.js`

5. Open your **Cursor AI** editor, highlight your empty `server.js` file, hit `Ctrl + K` (or open the Composer/Chat window Panel), copy-paste this rewritten SRS document along with the explicit instruction below, and watch the AI write it flawlessly:

   "As per the complete, professional Software Requirement Specification (SRS) provided above for the 'Gateway OTP' system, write the absolute, production-grade, modular, and highly secure Node.js Express application source code in server.js. Implement the full Redis-backed multi-channel smart routing logic, express route endpoints, error handling, rate-limiting counters, and cool-down lock implementations. Include clean, informative inline comments detailing the core caching operations and fail-safe streams."